Prefix Evaluation – Space Complexity

Program:

```
int prefixEvaluation(char *prefix)
{
    Stack st;
    int len = strlen(prefix);
    create(&st, len);

    int i = len - 1;

    int r, op1, op2;

    while (i >= 0 )
    {
        if (isOperand(prefix[i]) == 1)
        {
            push(&st, prefix[i] - '0');
        }
        else
        {
            op1 = pop(&st);

            op2 = pop(&st);


            switch (prefix[i])
            {
            case '+':
                r = op1 + op2;

                break;
            case '-':
                r = op1 - op2;
                break;
            case '*':
```

```
                r = op1 * op2;

                break;
            case '/':
                r = op1 / op2;

                break;
            case '^':
                r = pow(op1, op2);
                break;
            case '%':
                r = op1 % op2;
                break;
            }
            push(&st, r);
        }
        i--;
    }

    return pop(&st);
}
```

Space Complexity Analysis

Let $\boldsymbol{N}$ *be the length of the input string* $prefix$ *(calculated by* $strlen(prefix)$*).*

Input Space Complexity

The input space complexity refers to the memory space required to store the input data itself.

- ***String Input***: *The function takes a character pointer* $char\ *prefix$ *as an argument. The actual string passed to this function takes up memory proportional to the number of characters in the string.*

- ***Complexity***: $\boldsymbol{O}(\boldsymbol{N})$ *space is used by the caller to store the prefix expression in memory before passing it to the function.*

Auxiliary Space Complexity

The auxiliary space complexity refers to the temporary or extra space

used by the algorithm while it is executing, excluding the input data.

- ***Primitive Variables***: *The function declares a few primitive variables*: $len, i, r, op1$, *and* $op2$. *These are simple integer variables that take a constant amount of memory. This contributes* $\boldsymbol{O}(\mathbf{1})$ *to the space complexity.*

- ***The Stack*** $(\boldsymbol{st})$: *The primary consumer of extra memory is the stack. You call* $create(\&st, len)$; *, which dynamically allocates memory for the stack based on the length of the prefix string* (len). *In the absolute worst − case scenario (an expression with maximum possible operands), the stack will store items proportional to the size of the string.*

- ***Complexity***: $\boldsymbol{O}(\boldsymbol{N})$ *space is required for the stack allocation.*

Total Space Complexity

The total space complexity is the sum of the input space and the auxiliary space.

- *Input Space*: $\boldsymbol{O}(\boldsymbol{N})$
- *Auxiliary Space*: $\boldsymbol{O}(\boldsymbol{N})$
- ***Total Space Complexity***: $\boldsymbol{O}(\boldsymbol{N})$

****